Part 2: Pull Request Analysis

I selected “beetbox/beets” from the list of repositories provided

Link - beetbox/beets: music library manager and MusicBrainz tagger

PR 1: PR #4199 — Configure which fields are used to find duplicates

Link - Allow to configure which fields are used to find duplicates by jcassette · Pull Request #4199 · beetbox/beets

PR Summary

This PR enhances the Beets importer system by allowing the user to choose how duplicates are found while importing. Previously, Beets looked into a fixed list of fields (album name, artist, and so on) to see if the album was already in the library or not. However, in certain situations, people may want to store more than one copy of the album in a particular format, say MP3 and FLAC copies, or in different editions. In such scenarios, Beets may incorrectly identify them as duplicates and hence skip the importation. This PR introduces a new config setting named "duplicate_keys," using which the user can choose the fields they want duplicates to be compared on while identifying duplicates.

Technical Changes (files/components modified)

- beets/importer.py
  - uses the new duplicate_keys option to detect duplicates

Instead of this

```
duplicate_query = dbcore.AndQuery((
    dbcore.MatchQuery('albumartist', artist),
    dbcore.MatchQuery('album', album),
))
```

We are doing this

```
tmp_album = library.Album(lib, **info)
keys = config['import']['duplicate_keys']['album'].as_str_seq()
dup_query = library.Album.all_fields_query({
    key: tmp_album.get(key)
    for key in keys
})
```

- docs/changelog.rst
  - adds the entry about this new feature

```
Add the item fields ``bitrate_mode``, ``encoder_info`` and ``encoder_settings``.
Add query prefixes ``=`` and ``~``.
A new configuration option, :ref:`duplicate_keys`, lets you change which
fields the beets importer uses to identify duplicates.
```

- Config.rst
  - Documentation updated to add new importer configuration option duplicate_keys

Even though the defaults remain unchanged, the PR is implemented because it gives users the ability to customize duplicate detection, without breaking existing behavior.

```
There are two sub-values here: ``album`` and ``item``.
Each one is a list of field names; if an existing object (album or item) in
the library matches the new object on all of these fields, the importer will
consider it a duplicate.
```

Implementation Approach

The bulk of the change within the PR is the introduction of a new setting within the config named "duplicate_keys." The importer fetch this setting and applies it during the process of comparing for duplications. In the process of importing a new album, Beets searches for its existing duplicate in the file. In previous implementations, it did so using predefined fields. The new "duplicate_keys" setting allows the user to determine which keys to use.

If the user specifies duplicate_keys to include 'format' or 'bitrate,' this means that Beets will consider different formats for different file, so it will not incorrectly skip importing files wrongly classifies as duplicates. Finally, this Pull Request alters the code for importing to include these specified keys in its query to check for duplicate check query, which makes duplicate detection more accurate and user-controlled.

It improves the system since it provides more control to the user, who would have been forced to skip importing certain files in order to avoid inconsistencies in the program’s results.

Potential Impact

This PR affects the importing process, especially duplicate detection. It helps users who keep multiple versions of the same album. It reduces wrong duplicate skipping and allows a cleaner and more accurate library. Only the importer duplicate check logic changes, and most users will not notice any difference unless they enable the new config option.

PR 2: PR #3877 — Web readonly

Link - WebreadonlybyGrahamCobb · PullRequest#3877 · beetbox/beets

PR Summary

This PR adds a safety feature to Beets web plugin. The web plugin provides an API where users can view and manage their music library from a browser. But earlier, it allowed operations like PATCH and DELETE, which means a user could modify or even delete songs when using the web API. This PR introduces a new config option called readonly. When this option is turned on, the web API goes into read-only mode, meaning it only allows safe read operations like GET. It blocks any update/delete operations. This is helpful for users who just want to see their library online and want to avoid accidents. This makes the web plugin safer.

Technical Changes (files/components modified)

- Web plugin code (API request handling)
  - adds readonly check and blocks PATCH/DELETE when enabled

```
if get_method() == "DELETE":

    if app.config.get('READONLY', True):
        return flask.abort(405)
```

```
elif get_method() == "PATCH" and patchable:
    if app.config.get('READONLY', True):
        return flask.abort(405)
```

Line 465 in __init_.py

```
app.config['READONLY'] = self.config['readonly']
```

If the user has written readonly: yes, the plugin sets a "flag" (a digital switch) to True.

- Documentation
  - explains the new readonly option and behavior

```
**readonly**: If true, DELETE and PATCH operations are not allowed. Only GET is permitted.
Default: true.
```

```
Only allowed if ``readonly`` configuration option is set to ``no``.
```

```
``DELETE`` is only allowed if ``readonly`` configuration option is set to ``no``.
```

- docs/changelog.rst
  - adds changelog entry about readonly feature

```
:doc:`/plugins/web`: DELETE and PATCH methods are disallowed by default.
Set ``readonly: no`` web config option to enable them.
:bug:`3870`
```

By default now PATCH and DELETE are blocked

To allow them we much set readonly to no

- Tests
  - tests added to ensure PATCH/DELETE are blocked in readonly mode

Implementation Approach

In this PR, we introduce the configuration flag 'readonly' in the Beets web plugin. It's quite simple, when 'readonly' is activated, the web server must not accept any request that alters the library in any way. So, it checks the request method (it can be 'GET,' 'PATCH,' 'DELETE,' and so on). 'GET' is acceptable because it does not fetch any data, but 'PATCH' and 'DELETE' are not acceptable because they can alter or delete library information.

In the course of analyzing and debugging, it has been observed by the developer that it’s not sufficient to simply get and set app.config['READONLY'], which should read values from Beets’ configuration and get its own value using 'readonly'. A fix has been made so that now the web plugin respects the user’s config file. Tests have been included to ensure correct functioning of the 'readonly' feature.

Potential Impact

This PR changes the Beets web plugin API. After this PR, the web API is safer as it blocks all write operations unless the user disables readonly. That prevents the accidental deletion or unwanted change of the library through the web interface. Some users who relied on the PATCH/DELETE will have to disable readonly, but for the majority, it improves safety without harming normal browsing and their music library.